Libraries
Load Raw Data
Task 1: Data Preparation
Task 2: Customer Reporting Function
Task 3: Visualisations

Awesome Organic Food — Data Analysis Project

Student ID: 500778345 | Module: ASB-4012

30th March 2026

Libraries

```
# Core data manipulation
library(dplyr)      # filter, mutate, select, group_by, summarise
library(tidyr)      # unnest list columns
library(stringr)    # string splitting and manipulation

# Date handling
library(lubridate)  # date arithmetic, floor_date, difftime

# Visualisation
library(ggplot2)    # all charts and plots

# Reporting
library(knitr)      # kable() for tidy tables in the report
```

Load Raw Data

```
# Load the RDS file — raw_orders.rds is in the same directory as this RMD file, so no path needed.
raw_orders <- readRDS("raw_orders.rds")

# Quick sanity check: confirm the data loaded correctly
cat("Number of orders:  ", nrow(raw_orders), "\n")
```

```
## Number of orders:   200
```

```
cat("Number of columns: ", ncol(raw_orders), "\n")
```

```
## Number of columns:  10
```

```
cat("Column names:      ", paste(names(raw_orders), collapse = ", "), "\n")
```

```
## Column names:       Order_ID, Date, Discount_Code, Type, Customer, Customer_ID, Customer_DOB, Products, Quantities, Prices
```

```
cat("Date range:        ", as.character(min(raw_orders$Date)),
    "to", as.character(max(raw_orders$Date)), "\n")
```

```
## Date range:         2025-05-01 to 2025-08-31
```

```
cat("Order types:       ", paste(unique(raw_orders$Type), collapse = ", "), "\n")
```

```
## Order types:        Uber, Dine In
```

```
# Preview the first 6 rows (list columns show as abbreviated)
head(raw_orders)
```

	Order_ID <chr>	Date <date>	Discount_Code <chr>	Type <chr>	Customer <chr>
1	ORD0001	2025-08-19	NA	Uber	Charlotte Garcia
2	ORD0002	2025-06-03	OFF5	Uber	Abigail Allen
3	ORD0003	2025-08-22	NA	Dine In	Audrianna Chairez Esquivel
4	ORD0004	2025-08-14	NA	Uber	Audrianna Chairez Esquivel
5	ORD0005	2025-07-31	OFF20	Dine In	William Thomas
6	ORD0006	2025-07-08	OFF20	Uber	Mason Lee

6 rows | 1-6 of 11 columns

```
# Preview the list columns from the first order to confirm structure
cat("--- Order 1 Preview ---\n")
```

```
## --- Order 1 Preview ---
```

```
cat("Order ID:   ", raw_orders$Order_ID[1], "\n")
```

```
## Order ID:    ORD0001
```

```
cat("Customer:   ", raw_orders$Customer[1], "\n")
```

```
## Customer:    Charlotte Garcia
```

```
cat("Type:       ", raw_orders$Type[1], "\n")
```

```
## Type:        Uber
```

```
cat("Disc. Code: ", ifelse(is.na(raw_orders$Discount_Code[1]),
                           "None", raw_orders$Discount_Code[1]), "\n")
```

```
## Disc. Code:  None
```

```
cat("Products:   ", paste(raw_orders$Products[[1]],   collapse = ", "), "\n")
```

```
## Products:    Organic Salad, Lentil Soup, Veggie Wrap, Smoothie Bowl, Grilled Chicken
```

```
cat("Quantities: ", paste(raw_orders$Quantities[[1]], collapse = ", "), "\n")
```

```
## Quantities:  2, 1, 3, 3, 1
```

```
cat("Prices (£): ", paste(raw_orders$Prices[[1]],    collapse = ", "), "\n")
```

```
## Prices (£):  14.3, 14.74, 4.85, 8.96, 9.94
```

Task 1: Data Preparation

1A — Customers

```
# Extract one row per unique customer, split names and calculate age
customers <- raw_orders %>%
  distinct(Customer_ID, Customer, Customer_DOB) %>%
  mutate(
    name_parts  = str_split(Customer, " "),
    First_Name  = sapply(name_parts, function(x)
                    paste(x[-length(x)], collapse = " ")),
    Family_Name = sapply(name_parts, function(x)
                    x[length(x)])
  ) %>%
  mutate(
    Reference_Date = as.Date("2025-08-31"),
    Age = floor(
      as.numeric(difftime(Reference_Date,
                          as.Date(Customer_DOB),
                          units = "days")) / 365.25
    )
  ) %>%
  select(Customer_ID, First_Name, Family_Name, Age)

write.csv(customers, "customers.csv", row.names = FALSE)
cat("customers.csv saved successfully! Rows:", nrow(customers), "\n")
```

```
## customers.csv saved successfully! Rows: 30
```

```
kable(customers, caption = "Table 1: Customers Data Frame")
```

Table 1: Customers Data Frame

Customer_ID	First_Name	Family_Name	Age
CUST017	Charlotte	Garcia	32
CUST025	Abigail	Allen	34
CUST003	Audrianna Chairez	Esquivel	32
CUST012	William	Thomas	35
CUST022	Mason	Lee	33
CUST009	Olivia	Taylor	32
CUST010	Robert	Anderson	39
CUST005	Sarah	Williams	30
CUST002	Jane	Doe	40
CUST011	Sophia	Martinez	31
CUST001	John	Smith	35
CUST020	Alexander	Clark	36
CUST015	Mia	Harris	27
CUST018	Henry	Thompson	40
CUST026	Daniel	Young	40
CUST028	Matthew	Wright	36
CUST030	Joseph	Green	38
CUST007	Emily	Davis	28
CUST029	Scarlett	Scott	31
CUST004	Michael	Johnson	37
CUST019	Amelia	Robinson	27
CUST027	Ella	King	29
CUST013	Isabella	Jackson	29
CUST023	Evelyn	Walker	39
CUST006	David	Brown	41
CUST016	Lucas	Martin	34
CUST008	James	Wilson	33

Customer_ID	First_Name	Family_Name	Age
CUST014	Benjamin	White	38
CUST024	Ethan	Hall	26
CUST021	Harper	Lewis	30

1B — Items in Orders

```
# Unnest list columns so every product gets its own row
items_in_orders <- raw_orders %>%
  select(Order_ID, Products, Quantities, Prices) %>%
  unnest(cols = c(Products, Quantities, Prices)) %>%
  rename(
    Product    = Products,
    Quantity   = Quantities,
    Unit_Price = Prices
  )

write.csv(items_in_orders, "items_in_orders.csv", row.names = FALSE)
cat("items_in_orders.csv saved! Rows:", nrow(items_in_orders), "\n")
```

```
## items_in_orders.csv saved! Rows: 600
```

```
kable(head(items_in_orders, 15),
      caption = "Table 2: Items in Orders (first 15 rows)")
```

Table 2: Items in Orders (first 15 rows)

Order_ID	Product	Quantity	Unit_Price
ORD0001	Organic Salad	2	14.30
ORD0001	Lentil Soup	1	14.74
ORD0001	Veggie Wrap	3	4.85
ORD0001	Smoothie Bowl	3	8.96
ORD0001	Grilled Chicken	1	9.94
ORD0002	Smoothie Bowl	3	13.11
ORD0002	Avocado Toast	1	11.97
ORD0003	Organic Salad	2	11.28
ORD0003	Veggie Wrap	2	5.47
ORD0003	Fruit Salad	1	6.50
ORD0003	Tofu Stir Fry	1	9.41
ORD0003	Quinoa Bowl	2	11.26
ORD0003	Oat Pancakes	2	14.79
ORD0004	Avocado Toast	2	9.41
ORD0004	Grilled Chicken	2	3.52

```
expected_rows <- sum(sapply(raw_orders$Products, length))
actual_rows   <- nrow(items_in_orders)
cat("Expected rows:", expected_rows, "\n")
```

```
## Expected rows: 600
```

```
cat("Actual rows:  ", actual_rows,   "\n")
```

```
## Actual rows:   600
```

```
cat("Match:", ifelse(expected_rows == actual_rows,
                     "YES - data is correct!",
                     "NO  - something went wrong!"), "\n")
```

```
## Match: YES - data is correct!
```

1C — Orders (with Financial Calculations)

```
# Full financial calculation pipeline for every order
orders <- raw_orders %>%
  mutate(
    Sub_Total_Before_Discount = mapply(
      function(q, p) sum(q * p), Quantities, Prices)
  ) %>%
  mutate(Total_Items = sapply(Quantities, sum)) %>%
  mutate(Large_Discount_Rate = ifelse(Total_Items > 3, 0.10, 0)) %>%
  mutate(
    Code_Discount_Rate = case_when(
      Discount_Code == "OFF5"  ~ 0.05,
      Discount_Code == "OFF20" ~ 0.20,
      TRUE                     ~ 0.00
    )
  ) %>%
  mutate(Final_Discount_Rate = pmax(Large_Discount_Rate, Code_Discount_Rate)) %>%
  mutate(
    Discount_Code = case_when(
      Final_Discount_Rate == 0.20 ~ "OFF20",
      Final_Discount_Rate == 0.10 ~ "LARGE10",
      Final_Discount_Rate == 0.05 ~ "OFF5",
      TRUE                        ~ NA_character_
    )
  ) %>%
  mutate(
    Discount_Value            = round(Sub_Total_Before_Discount *
                                      Final_Discount_Rate, 2),
    Sub_Total_After_Discount  = round(Sub_Total_Before_Discount -
                                      Discount_Value, 2),
    VAT                       = round(Sub_Total_After_Discount * 0.10, 2),
    Uber_Fee                  = ifelse(Type == "Uber", 1.99, 0.00),
    Final_Total               = round(Sub_Total_After_Discount +
                                      VAT + Uber_Fee, 2),
    Sub_Total_Before_Discount = round(Sub_Total_Before_Discount, 2)
  ) %>%
  select(
    Order_ID, Customer_ID, Type, Discount_Code, Date,
    Uber_Fee, Sub_Total_Before_Discount, Discount_Value,
    Sub_Total_After_Discount, VAT, Final_Total
  )

write.csv(orders, "orders.csv", row.names = FALSE)
cat("orders.csv saved successfully! Rows:", nrow(orders), "\n")
```

```
## orders.csv saved successfully! Rows: 200
```

```
cat("LARGE10:", sum(orders$Discount_Code == "LARGE10", na.rm = TRUE),
    "| OFF5:", sum(orders$Discount_Code == "OFF5", na.rm = TRUE),
    "| OFF20:", sum(orders$Discount_Code == "OFF20", na.rm = TRUE),
    "| None:", sum(is.na(orders$Discount_Code)), "\n")
```

```
## LARGE10: 121 | OFF5: 13 | OFF20: 34 | None: 32
```

```
kable(head(orders, 10),
      caption = "Table 3: Orders Data Frame (first 10 rows)",
      digits  = 2)
```

Table 3: Orders Data Frame (first 10 rows)

Order_ID	Customer_ID	Type	Discount_Code	Date	Uber_Fee	Sub_Total_Before_Discount	Discount_Value	Sub_Total_After_Discount	VAT	Final_Tot
ORD0001	CUST017	Uber	LARGE10	2025-08-19	1.99	94.71	9.47	85.24	8.52	95.7
ORD0002	CUST025	Uber	LARGE10	2025-06-03	1.99	51.30	5.13	46.17	4.62	52.7

Order_ID	Customer_ID	Type	Discount_Code	Date	Uber_Fee	Sub_Total_Before_Discount	Discount_Value	Sub_Total_After_Discount	VAT	Final_Tot
ORD0003	CUST003	Dine In	LARGE10	2025-08-22	0.00	101.51	10.15	91.36	9.14	100.5
ORD0004	CUST003	Uber	LARGE10	2025-08-14	1.99	41.35	4.14	37.21	3.72	42.9
ORD0005	CUST012	Dine In	OFF20	2025-07-31	0.00	16.40	3.28	13.12	1.31	14.4
ORD0006	CUST022	Uber	OFF20	2025-07-08	1.99	7.97	1.59	6.38	0.64	9.0
ORD0007	CUST025	Uber	LARGE10	2025-05-29	1.99	74.18	7.42	66.76	6.68	75.4
ORD0008	CUST009	Dine In	LARGE10	2025-07-12	0.00	66.23	6.62	59.61	5.96	65.5
ORD0009	CUST010	Dine In	LARGE10	2025-08-19	0.00	86.28	8.63	77.65	7.77	85.4
ORD0010	CUST005	Uber	LARGE10	2025-07-23	1.99	91.53	9.15	82.38	8.24	92.6

```
i            <- 1
check_order  <- raw_orders[i, ]
check_result <- orders[i, ]
manual_subtotal <- sum(check_order$Quantities[[1]] * check_order$Prices[[1]])
manual_discount <- check_result$Discount_Value
manual_after    <- manual_subtotal - manual_discount
manual_vat      <- round(manual_after * 0.10, 2)
manual_uber     <- ifelse(check_order$Type == "Uber", 1.99, 0.00)
manual_total    <- round(manual_after + manual_vat + manual_uber, 2)

cat("=== Manual Verification for", check_result$Order_ID, "===\n")
```

```
## === Manual Verification for ORD0001 ===
```

```
cat("Sub-Total Before Discount:", round(manual_subtotal, 2), "\n")
```

```
## Sub-Total Before Discount: 94.71
```

```
cat("Discount Code Applied:    ",
    ifelse(is.na(check_result$Discount_Code),
           "None", check_result$Discount_Code), "\n")
```

```
## Discount Code Applied:     LARGE10
```

```
cat("Final Total (manual):     ", manual_total, "\n")
```

```
## Final Total (manual):      95.75
```

```
cat("Final Total (stored):     ", check_result$Final_Total, "\n")
```

```
## Final Total (stored):      95.75
```

```
cat("Match:", ifelse(manual_total == check_result$Final_Total,
                     "YES - correct!", "NO - error!"), "\n")
```

```
## Match: YES - correct!
```

Task 2: Customer Reporting Function

```r
# ── customer_report() ──────────────────────────────────────────────────────
# Generates a summary report for a given customer over a date range.
# Parameters:
#   customer_id : unique customer ID string (e.g. "CUST001")
#   start_date  : start date string "YYYY-MM-DD" (inclusive)
#   end_date    : end date string "YYYY-MM-DD"   (inclusive)
#   customers, orders and items_in_orders files are in the global environment.
# ───────────────────────────────────────────────────────────────────────────

customer_report <- function(customer_id, start_date, end_date) {

  start <- as.Date(start_date)
  end   <- as.Date(end_date)

  # Look up customer record
  cust <- customers %>% filter(Customer_ID == customer_id)

  # Filter orders within the date range for this customer
  cust_orders <- orders %>%
    filter(Customer_ID == customer_id,
           as.Date(Date) >= start,
           as.Date(Date) <= end)

  # Handle no orders found
  if (nrow(cust_orders) == 0) {
    if (nrow(cust) == 0) {
      cat("Customer ID '", customer_id,
          "' was not found in the system.\n", sep = "")
    } else {
      cat("No orders found for customer '", customer_id,
          "' between ", start_date, " and ", end_date, ".\n", sep = "")
    }
    return(invisible(NULL))
  }

  # Build full name
  full_name <- paste(cust$First_Name, cust$Family_Name)

  # Get items for these orders only
  cust_items <- items_in_orders %>%
    filter(Order_ID %in% cust_orders$Order_ID)

  # Order type percentage breakdown
  type_breakdown <- cust_orders %>%
    count(Type) %>%
    mutate(Percentage = round(n / sum(n) * 100, 1)) %>%
    rename(Count = n)

  # Product percentage breakdown by total quantity ordered
  product_breakdown <- cust_items %>%
    group_by(Product) %>%
    summarise(Total_Qty = sum(Quantity), .groups = "drop") %>%
    mutate(Percentage = round(Total_Qty / sum(Total_Qty) * 100, 1)) %>%
    arrange(desc(Percentage))

  # Print the report
  cat("================================================\n")
  cat(" CUSTOMER REPORT\n")
  cat("================================================\n")
  cat(" Customer ID:   ", customer_id,               "\n")
  cat(" Full Name:     ", full_name,                  "\n")
  cat(" Age:           ", cust$Age, "years",          "\n")
  cat(" Period:        ", start_date, "to", end_date, "\n")
  cat("------------------------------------------------\n")
  cat(" ORDER SUMMARY\n")
  cat("------------------------------------------------\n")
  cat(" Number of Orders:", nrow(cust_orders),                        "\n")
  cat(" Mean Total:      £", round(mean(cust_orders$Final_Total), 2),   "\n")
  cat(" Median Total:    £", round(median(cust_orders$Final_Total), 2), "\n")
  cat("------------------------------------------------\n")
  cat(" ORDER TYPE BREAKDOWN\n")
  cat("------------------------------------------------\n")
  for (i in seq_len(nrow(type_breakdown))) {
    cat(" ", type_breakdown$Type[i], ":",
        type_breakdown$Count[i], "orders",
        paste0("(", type_breakdown$Percentage[i], "%)"), "\n")
  }
  cat("------------------------------------------------\n")
```

```
  cat(" PRODUCT BREAKDOWN (by quantity ordered)\n")
  cat("-----------------------------------------------\n")
  for (i in seq_len(nrow(product_breakdown))) {
    cat(" ", product_breakdown$Product[i], ":",
        product_breakdown$Total_Qty[i], "units",
        paste0("(", product_breakdown$Percentage[i], "%)"), "\n")
  }
  cat("===============================================\n")
}
```

```
# Demo 1: Valid customer with orders — full report
customer_report("CUST001", "2025-05-01", "2025-08-31")
```

```
## ===============================================
##  CUSTOMER REPORT
## ===============================================
##  Customer ID:    CUST001
##  Full Name:      John Smith
##  Age:            35 years
##  Period:         2025-05-01 to 2025-08-31
## -----------------------------------------------
##  ORDER SUMMARY
## -----------------------------------------------
##  Number of Orders: 12
##  Mean Total:      £ 59.14
##  Median Total:    £ 61.7
## -----------------------------------------------
##  ORDER TYPE BREAKDOWN
## -----------------------------------------------
##   Dine In : 9 orders (75%)
##   Uber : 3 orders (25%)
## -----------------------------------------------
##  PRODUCT BREAKDOWN (by quantity ordered)
## -----------------------------------------------
##   Smoothie Bowl : 13 units (15.9%)
##   Beetroot Salad : 11 units (13.4%)
##   Avocado Toast : 10 units (12.2%)
##   Lentil Soup : 8 units (9.8%)
##   Quinoa Bowl : 8 units (9.8%)
##   Organic Salad : 7 units (8.5%)
##   Green Juice : 6 units (7.3%)
##   Tofu Stir Fry : 5 units (6.1%)
##   Acai Bowl : 4 units (4.9%)
##   Vegan Burger : 4 units (4.9%)
##   Fruit Salad : 2 units (2.4%)
##   Grilled Chicken : 1 units (1.2%)
##   Herbal Tea : 1 units (1.2%)
##   Oat Pancakes : 1 units (1.2%)
##   Veggie Wrap : 1 units (1.2%)
## ===============================================
```

```
# Demo 2: Valid customer but no orders in range
customer_report("CUST001", "2024-01-01", "2024-01-31")
```

```
## No orders found for customer 'CUST001' between 2024-01-01 and 2024-01-31.
```

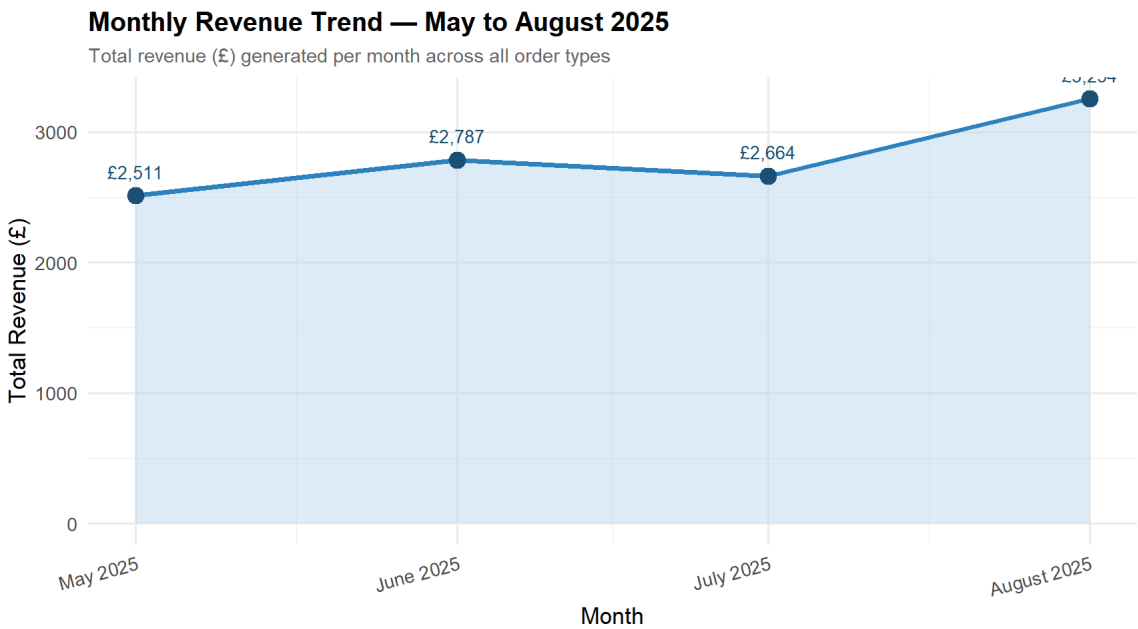
```
# Demo 3: Customer ID that does not exist
customer_report("CUST999", "2025-05-01", "2025-08-31")
```

```
## Customer ID 'CUST999' was not found in the system.
```

Task 3: Visualisations

Chart 1 — Monthly Revenue Trend

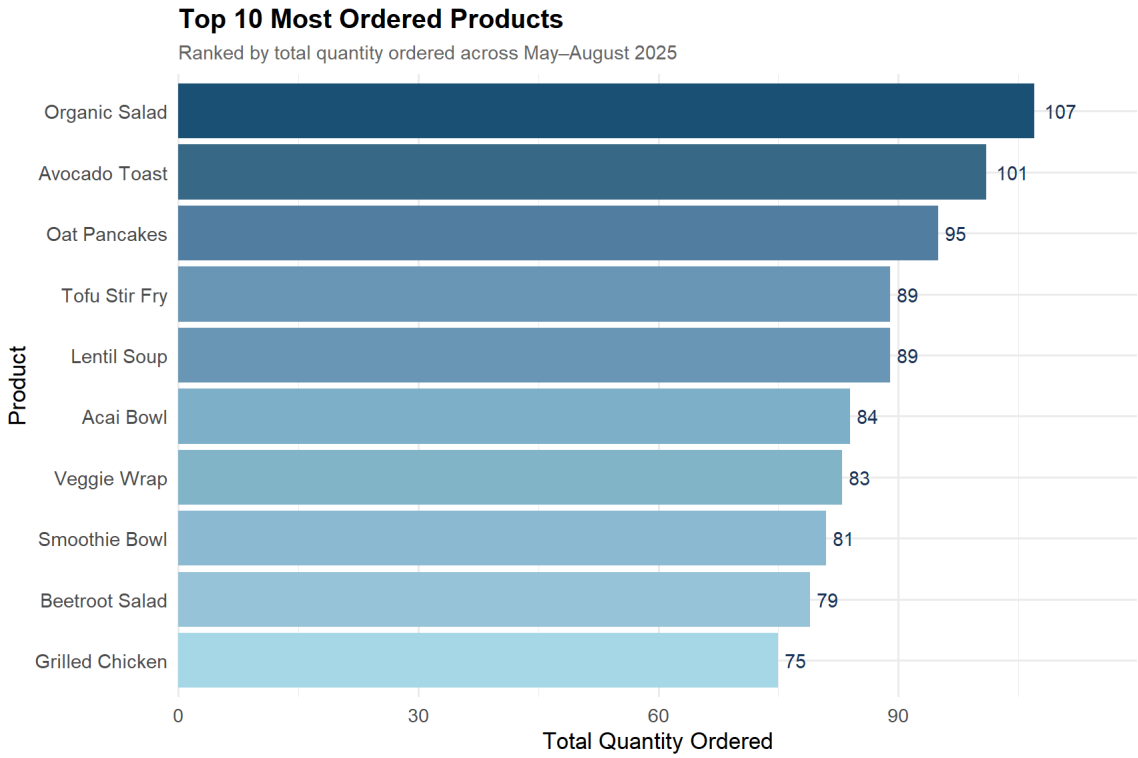
```r
# ── Data preparation ──────────────────────────────────────────────────────
# Group orders by month and sum the Final_Total for each month.
# floor_date() rounds each date DOWN to the 1st of its month.
# This groups all orders in the same month together under one label.

monthly_revenue <- orders %>%
  mutate(Month = floor_date(as.Date(Date), "month")) %>%
  group_by(Month) %>%
  summarise(Total_Revenue = sum(Final_Total), .groups = "drop")

# ── Chart ─────────────────────────────────────────────────────────────────
ggplot(monthly_revenue, aes(x = Month, y = Total_Revenue)) +

  # Shaded area under the line for visual impact
  geom_area(fill = "#AED6F1", alpha = 0.4) +

  # Main trend line
  geom_line(color = "#2E86C1", linewidth = 1.4) +

  # Data point markers at each month
  geom_point(color = "#1A5276", size = 4) +

  # Revenue labels above each point
  geom_text(aes(label = paste0("£", formatC(Total_Revenue,
                                            format = "f",
                                            digits = 0,
                                            big.mark = ",")),
            vjust = -1.2, size = 3.5, color = "#1A5276") +

  # Axis and title labels
  labs(
    title    = "Monthly Revenue Trend — May to August 2025",
    subtitle = "Total revenue (£) generated per month across all order types",
    x        = "Month",
    y        = "Total Revenue (£)"
  ) +

  # Format x-axis to show month name and year
  scale_x_date(date_labels = "%B %Y", date_breaks = "1 month") +

  # Theme
  theme_minimal(base_size = 13) +
  theme(
    plot.title    = element_text(face = "bold", size = 15),
    plot.subtitle = element_text(color = "grey40", size = 11),
    axis.text.x   = element_text(angle = 15, hjust = 1)
  )
```



Interpretation: The chart shows total revenue generated each month from May to August 2025. Revenue is displayed as a trend line with exact monthly totals labelled above each data point. This allows the restaurant owner to quickly identify which months performed strongest and whether revenue is growing or declining over the four-month period. For example, any month showing a notable spike may indicate a successful promotion or seasonal demand, while a dip could signal a period requiring attention. The owner can use this insight to plan staffing, stock purchasing, and marketing campaigns around peak and low revenue periods.

Chart 2 — Top 10 Most Ordered Products

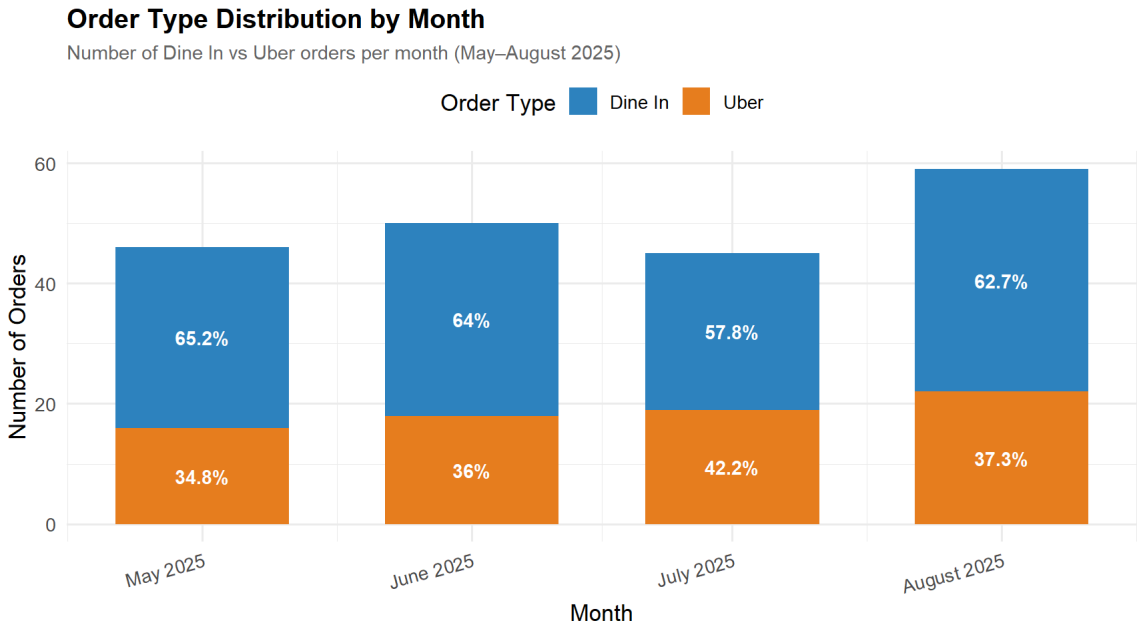
```
# ── Data preparation ──────────────────────────────────────────────────────
# Sum total quantity ordered for every product across all orders,
# Only the top 10 by volume is kept.

top_products <- items_in_orders %>%
  group_by(Product) %>%
  summarise(Total_Qty = sum(Quantity), .groups = "drop") %>%
  arrange(desc(Total_Qty)) %>%
  slice_head(n = 10) %>%

  # reorder() sorts bars from highest to lowest when coord_flip() is used
  mutate(Product = reorder(Product, Total_Qty))

# ── Chart ─────────────────────────────────────────────────────────────────
ggplot(top_products, aes(x = Product, y = Total_Qty, fill = Total_Qty)) +

  # Horizontal bars
  geom_col(show.legend = FALSE) +

  # Quantity labels at the end of each bar
  geom_text(aes(label = Total_Qty),
            hjust = -0.3, size = 3.8, color = "#1D3557") +

  # Flip to horizontal layout for easier reading of product names
  coord_flip(clip = "off") +

  # Colour gradient: lighter = fewer orders, darker = more orders
  scale_fill_gradient(low = "#A8D8EA", high = "#1A5276") +

  # Extend x-axis slightly so labels are not clipped
  scale_y_continuous(expand = expansion(mult = c(0, 0.12))) +

  labs(
    title    = "Top 10 Most Ordered Products",
    subtitle = "Ranked by total quantity ordered across May–August 2025",
    x        = "Product",
    y        = "Total Quantity Ordered"
  ) +

  theme_minimal(base_size = 13) +
  theme(
    plot.title    = element_text(face = "bold", size = 15),
    plot.subtitle = element_text(color = "grey40", size = 11),
    axis.text.y   = element_text(size = 11)
  )
```



Interpretation: This chart ranks the ten best-selling products by total quantity ordered between May and August 2025. The colour gradient reinforces the ranking — darker bars represent higher-volume products. This insight is directly useful for the restaurant owner in several ways: top-selling items should always be well-stocked and prominently featured on the menu, while lower-ranked products may warrant a review of pricing or presentation. If any product consistently underperforms, the owner may consider replacing it with a new offering to refresh the menu and attract more orders.

Chart 3 — Order Type Distribution by Month

```
# ── Data preparation ──────────────────────────────────────────────────────
# Count the number of Dine In and Uber orders placed each month.
# This lets us compare channel preferences over time.

order_type_monthly <- orders %>%
  mutate(Month = floor_date(as.Date(Date), "month")) %>%
  count(Month, Type) %>%

  # Calculate percentage of each type within each month
  group_by(Month) %>%
  mutate(
    Total      = sum(n),
    Percentage = round(n / Total * 100, 1)
  ) %>%
  ungroup()

# ── Chart ─────────────────────────────────────────────────────────────────
ggplot(order_type_monthly,
       aes(x = Month, y = n, fill = Type)) +

  # Stacked bars — each bar = one month, split by order type
  geom_col(position = "stack", width = 20) +

  # Percentage labels inside each bar segment (only show if segment is tall enough)
  geom_text(aes(label = ifelse(n > 3,
                               paste0(Percentage, "%"), "")),
            position = position_stack(vjust = 0.5),
            size = 3.8, color = "white", fontface = "bold") +

  # Custom colours for each order type
  scale_fill_manual(
    values = c("Dine In" = "#2E86C1", "Uber" = "#E67E22"),
    name   = "Order Type"
  ) +

  # Format x-axis as month names
  scale_x_date(date_labels = "%B %Y", date_breaks = "1 month") +

  labs(
    title    = "Order Type Distribution by Month",
    subtitle = "Number of Dine In vs Uber orders per month (May–August 2025)",
    x        = "Month",
    y        = "Number of Orders",
    fill     = "Order Type"
  ) +

  theme_minimal(base_size = 13) +
  theme(
    plot.title    = element_text(face = "bold", size = 15),
    plot.subtitle = element_text(color = "grey40", size = 11),
    axis.text.x   = element_text(angle = 15, hjust = 1),
    legend.position = "top"
  )
```



Interpretation: This stacked bar chart compares the number of Dine In and Uber orders placed each month from May to August 2025. Each bar represents one month, split proportionally between the two order types, with percentage labels shown inside each segment. This view helps the restaurant owner understand how customers prefer to receive their orders and whether that preference changes across the summer period. A growing Uber share may suggest the owner should negotiate better delivery terms or optimise packaging for delivery, while a strong Dine In presence reinforces the value of maintaining a welcoming in-restaurant experience.